

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU

BANCO DE DADOS

Gustavo Luigi Lasalvia de Carvalho

Jefferson Feitosa da Silva Cardoso

Marcos Vinicius Portela de Lira

Pedro José Santiago Marcelino

Raphael Vilas Boas Scanoni

Vanessa Priscila Gameiro Marques

Vitor Luis Gomes Vieira

SISTEMA DE CONTROLE DE ESTOQUE

RECIFE

2026

A partir de conversas e observações realizadas pelo grupo, chegou-se em consenso que a temática escolhida para desenvolver o projeto da disciplina de Banco de Dados seria o “Sistema de Controle de Estoque”, pois um dos integrantes do grupo trabalha diretamente com esse conteúdo. Dessa forma, pudemos nos aprofundar, questionar e entender a nossa situação-problema e, também, refletir quais requisitos seriam necessários para o desenvolvimento do nosso trabalho.

Com base nisso, buscou-se nesse projeto solucionar uma situação recorrente em almoxarifados: a não digitalização dos processos que envolvem o local. Esse problema acarreta na falta de precisão sobre a quantidade do estoque existente, o qual pode diferir na quantidade real de produtos existentes e na comparação com a realidade. Assim, objetivou-se realizar um balanço de estoque, de maneira semestral, comparando a quantidade de produtos físicos existentes com a quantidade registrada no sistema, de maneira a deixar os dois alinhados. Para realizar o registro desses dados, utilizou-se o banco de dados MySQL, tendo como referência os funcionários do almoxarifado que utilizam o sistema regularmente.

Para isso, com o intuito de melhor desenvolvimento do sistema, elencou-se como os principais requisitos funcionais: i. O sistema deve permitir o cadastro de produtos, contendo informações como identificador único, nome, descrição, quantidade de unidades e unidade de medida; ii. O sistema deve permitir o registro de pedidos no almoxarifado, contendo os seguintes campos: identificação única, nome do produto, solicitante, responsável a autorizar o pedido e setor; iii. O sistema deve permitir a consulta da quantidade de produtos registrada no estoque; iv. O sistema deve permitir o registro da quantidade física de cada produto durante o balanço de estoque; v. O sistema deve comparar a quantidade física dos produtos com a quantidade registrada no banco de dados; vi. O sistema deve identificar e apresentar divergências entre as quantidades físicas e as quantidades registradas no sistema; vii. O sistema deve permitir a atualização das quantidades em estoque após a conferência e a confirmação das divergências encontradas; viii. O sistema deve gerar relatórios do balanço de estoque, apresentando as quantidades registradas, as quantidades físicas e as divergências identificadas; ix. O sistema deve permitir que mais de um produto seja cadastrado em um pedido.

Ademais, para a estruturação das tabelas e implementação das entidades no banco de dados, separou-se da seguinte forma: i. nome: produtos / finalidade: armazenar os produtos e seus atributos / principais atributos: id_produto,

qtd_produto, nome_produto, produto_pedido; ii. nome: pedidos / finalidade: armazenar os pedidos enviados ao almoxarifado / principais atributos: id_pedido, setor, autorizador, solicitante; iii. nome: funcionário / finalidade: registrar os funcionários do hospital. / principais atributos: id_funcionario, cargo, nome_funcionario, pode_autorizar; iv. nome: pedidos_produtos / finalidade: relacionar as tabelas de produtos e pedidos em uma tabela separada. / principais atributos: id_pedido, id_produto, qtd_solicitada.

Também, a partir do dito anteriormente, determinou-se alguns atributos, chaves e relacionamentos considerados necessários para a construção do banco de dados e fez-se o modelo conceitual, conforme listado abaixo:

PRODUTOS

---

id_produto PK UNICO NOT NULL (PK pois é o id do produto, ÚNICO pois cada produto é único, e NOT NULL pois sempre deve existir)
qtd_produto
nome_produto NOT NULL (NOT NULL pois sempre deve existir)
produto_pedido FK (FK para se relacionar com outras tabelas [PEDIDOS])

PEDIDOS

---

id_pedido PK UNICO NOT NULL (PK pois é o id do pedido, UNICO pois cada produto é único, e NOT NULL pois sempre deve existir)
setor
qtd_produto_pedido NOT NULL (NOT NULL pois sempre deve existir)
id_solicitante FK (FK para se relacionar com outras tabelas [FUNCIONARIOS])
id_autorizador NOT NULL FK (FK para se relacionar com outras tabelas [FUNCIONARIOS])

FUNCIONARIOS

---

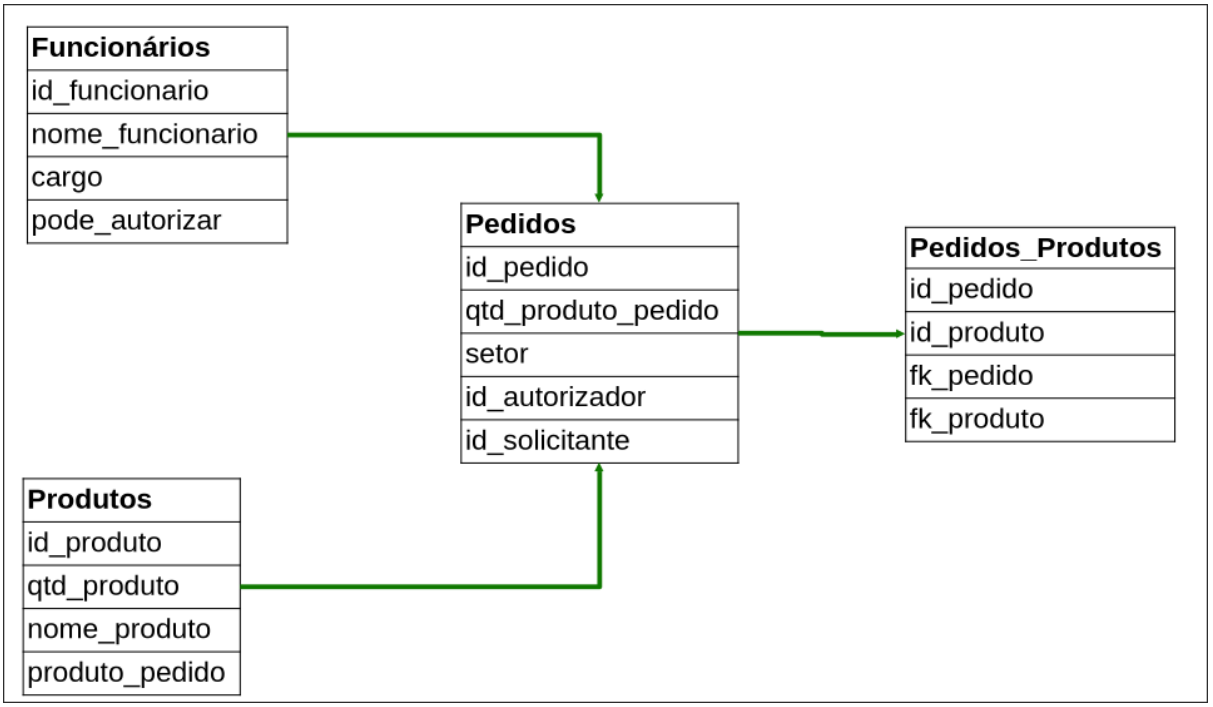
id_funcionario PK UNICO NOT NULL (PK pois é o id do produto, UNICO pois cada produto é único, e NOT NULL pois sempre deve existir)
nome_funcionario NOT NULL (NOT NULL pois sempre deve existir)

cargo NOT NULL (NOT NULL pois sempre deve existir)
pode_autorizar NOT NULL (NOT NULL pois sempre deve existir)

PEDIDOS_PRODUTOS
---
id_produto PK NOT NULL (PK pois é o id do produto, NOT NULL pois sempre deve existir)
id_pedido PK NOT NULL (PK pois é o id do pedido, NOT NULL pois sempre deve existir)
qtd_solicitada NOT NULL (NOT NULL pois sempre deve existir)
fk_pedido FK (FK para se relacionar com outras tabelas [PEDIDOS])
fk_produto FK (FK para se relacionar com outras tabelas [PRODUTOS])

De acordo com o visto acima, foi estimado os seguintes relacionamentos entre as entidades:

FUNCIONARIO 1 —> X PEDIDOS
PEDIDO 1 —> X PRODUTOS



Modelo lógico (diagrama)

De acordo com o que foi apresentado, justifica–se que o banco de dados pode evitar os seguintes problemas estipulados: i. Evitar redundâncias desnecessárias, cada entidade possui os atributos específicos relacionados ao objetivo da tabela, evitando que o mesmo dado exista em duas ou mais tabelas, economizando espaço e facilitando a visualização; ii. As informações foram separadas entre as tabelas através de uma análise de quais dados seriam importantes para cada entidade e, desse modo, otimizando o controle de dados do nosso estoque; iii. Como citado no item i, os atributos foram definidos com o intuito de, também, evitar repetições em duas ou mais tabelas, a PK(Primary Key, chave primária) é única e estabelece relação com as tabelas que se comunicam com as FK(Foreign Key, chave estrangeira) com cada uma destas apontando para a PK da entidade, auxiliando na organização, visto que é possível transformar tabelas grandes em tabelas menores e mais lógicas, mitigando a desorganização e economizando tempo; iv. Uma forma de modelagem do banco de dados vista em sala de aula foi as tabelas de junções, feitas para fazer a junção de duas entidades distintas, sendo usadas em duas tabelas (pedidos_produtos e funcionarios_pedidos).

Com base no que foi mostrado, foi feita a implementação do banco de dados via MySQL Workbench, de acordo com os comandos abaixo:

```
CREATE DATABASE estoque_almoxarifado;

USE estoque_almoxarifado;

CREATE TABLE IF NOT EXISTS funcionarios (
    id_funcionario INT PRIMARY KEY,
    nome_funcionario VARCHAR(100) NOT NULL,
    cargo VARCHAR(100) NOT NULL,
    pode_autorizar BOOLEAN NOT NULL
);

CREATE TABLE IF NOT EXISTS pedidos (
    id_pedido INT PRIMARY KEY,
    setor VARCHAR(50),
    id_solicitante INT NOT NULL,
    id_autorizador INT,

        CONSTRAINT fk_pedido_solicitante FOREIGN KEY (id_solicitante) REFERENCES funcionarios(id_funcionario),
        CONSTRAINT fk_pedido_autorizador FOREIGN KEY (id_autorizador) REFERENCES funcionarios(id_funcionario)
);

CREATE TABLE IF NOT EXISTS produtos (
    id_produto INT PRIMARY KEY,
    qtd_produto INT,
    nome_produto VARCHAR(150) NOT NULL
);

CREATE TABLE IF NOT EXISTS pedidos_produtos(
    id_pedido INT NOT NULL,
    id_produto INT NOT NULL,
```

```
    qtd_solicitada INT NOT NULL,
    PRIMARY KEY (id_pedido, id_produto),
        CONSTRAINT fk_pp_pedido FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
        CONSTRAINT fk_pp_produto FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
);
```

Após a criação do banco de dados, foi inserido alguns dados fictícios para popular o banco:

```
INSERT INTO funcionarios VALUES
(1, "João", "Encarregado", true),
(2, "Maria", "Funcionária do almoxarifado", false),
(3, "Carlos", "Cirurgião", false),
(4, "José", "Líder do setor 1", true),
(5, "Fernanda", "Enfermeira", false);

INSERT INTO produtos VALUES
(1, 23, "Seringa 100ui"),
(2, 12, "Band-aid (Caixa de 100 unidades)"),
(3, 3, "Soro Fisiológico (500ml)"),
(4, 48, "Luva de látex"),
(5, 6, "Máscara descartável");

INSERT INTO pedidos VALUES
(1, "Pronto-Socorro", 3, 1),
(2, "Sala 12", 5, 4),
(3, "Administração", 2, 1),
(4, "Sala de Radiologia", 5, 4),
(5, "Sala 15", 3, 1);

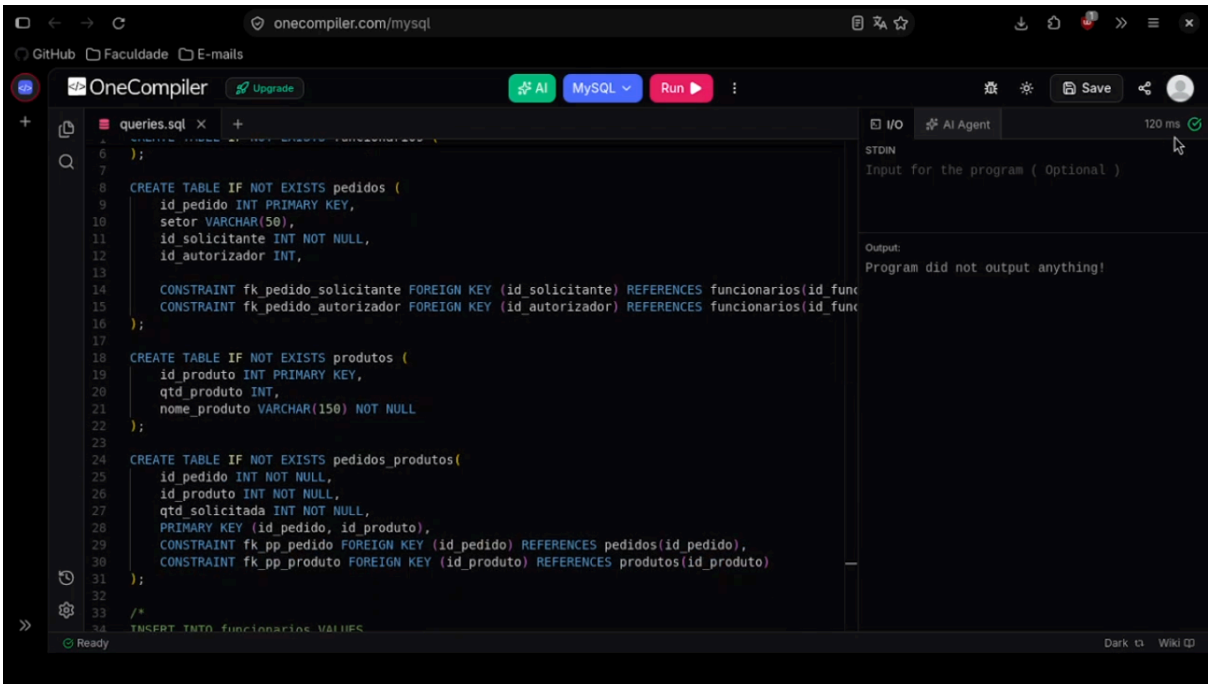
INSERT INTO pedidos_produtos VALUES
(1, 2, 10),
```

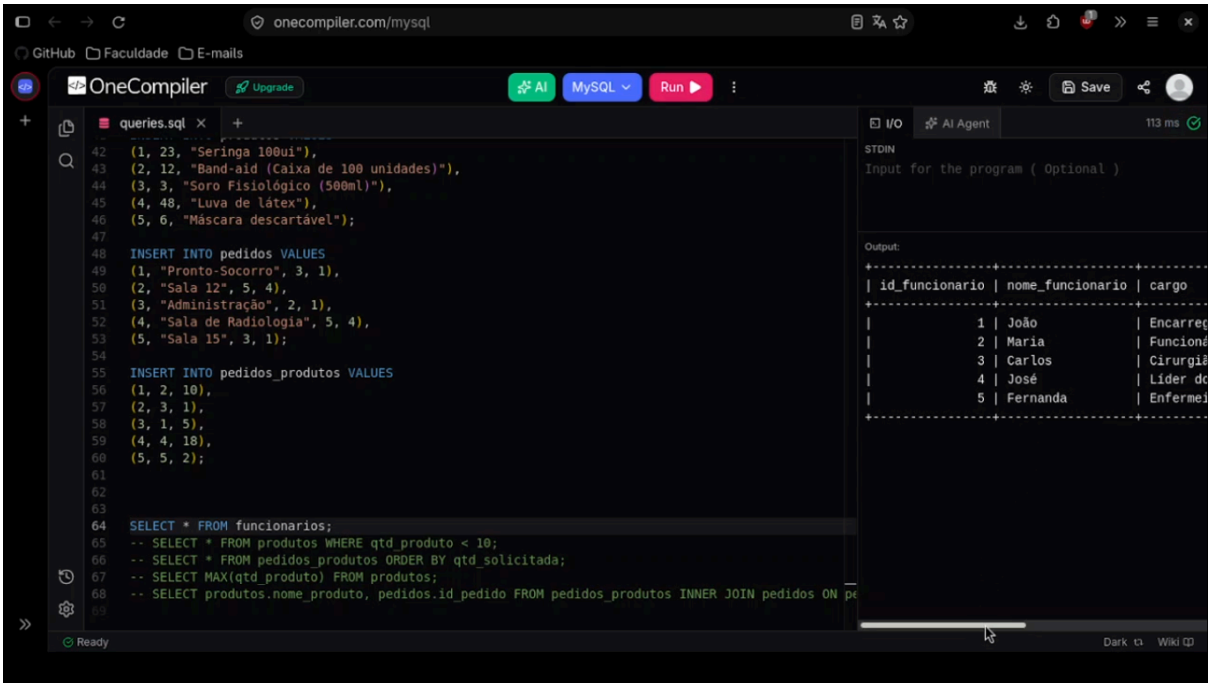
(2, 3, 1),
(3, 1, 5),
(4, 4, 18),
(5, 5, 2);

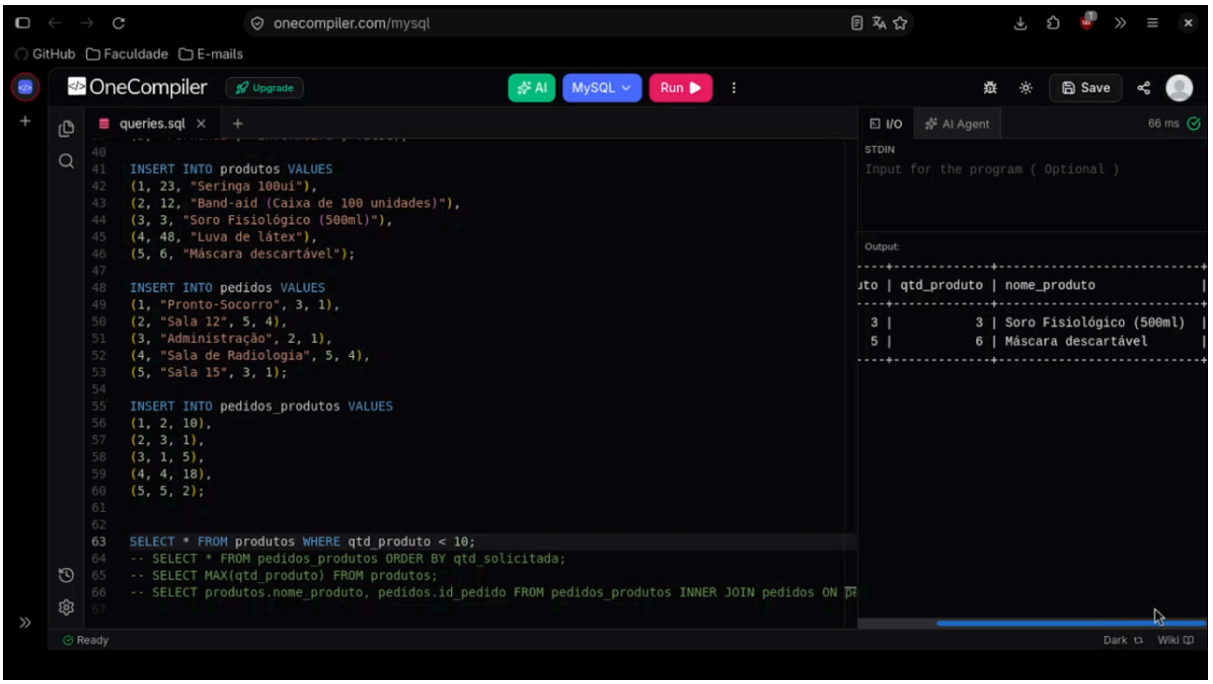
Para a realização das consultas, foram usados os seguintes comandos:

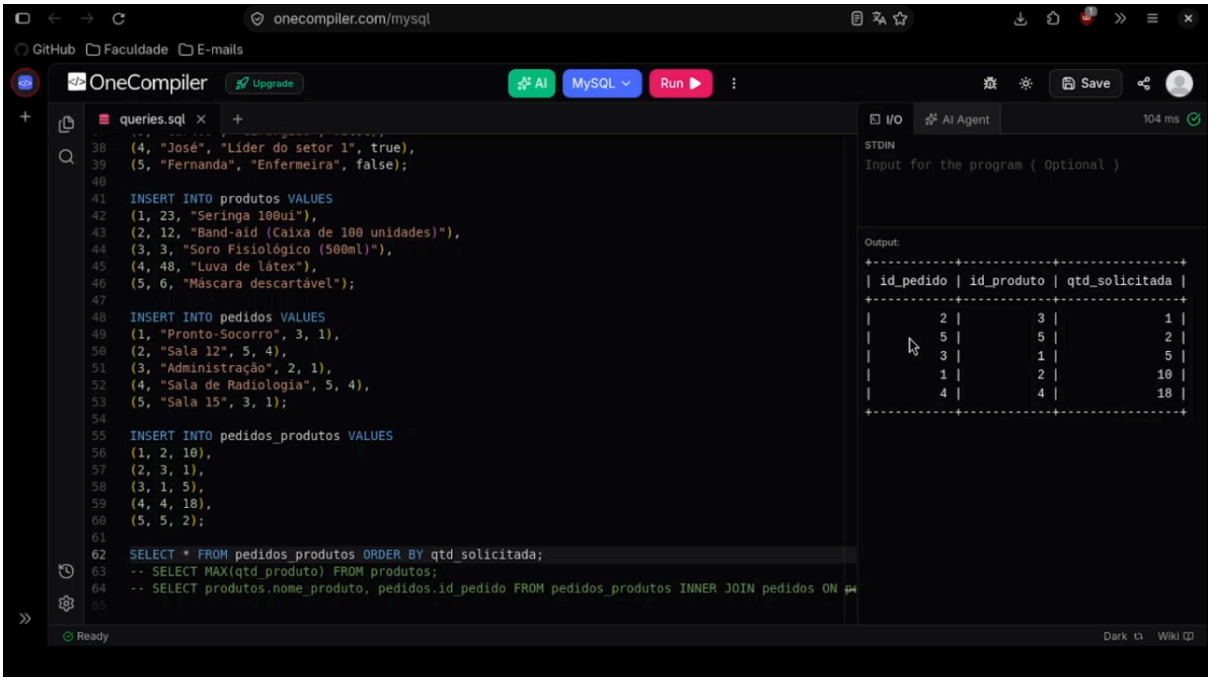
Consulta 1 listagem SELECT * FROM funcionarios;
Consulta 2 filtragem SELECT * FROM produtos WHERE qtd_produto < 10;
Consulta 3 ordenação SELECT * FROM pedidos_produtos ORDER BY qtd_solicitada;
Consulta 4 agregação SELECT MAX(qtd_produto) FROM produtos;
Consulta 5 junção SELECT produtos.nome_produto, pedidos.id_pedido FROM pedidos_produtos INNER JOIN pedidos ON pedidos_produtos.id_pedido = pedidos.id_pedido INNER JOIN produtos ON pedidos_produtos.id_produto = produtos.id_produto;

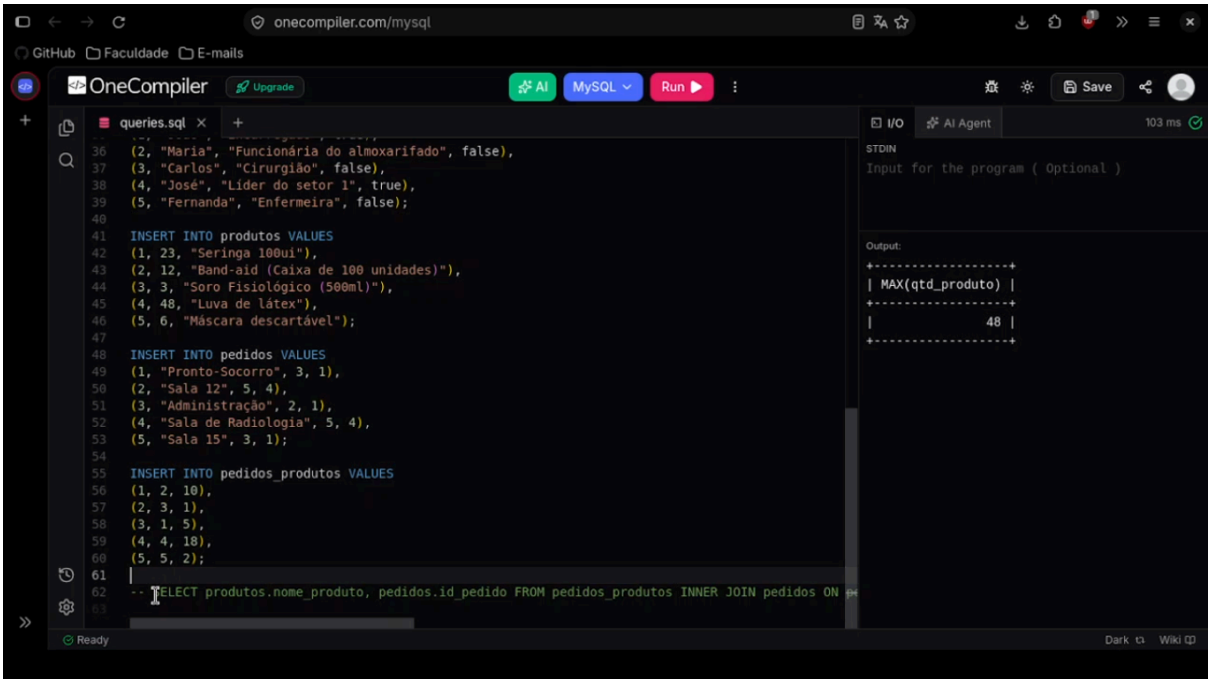
RESULTADOS E EVIDÊNCIAS:

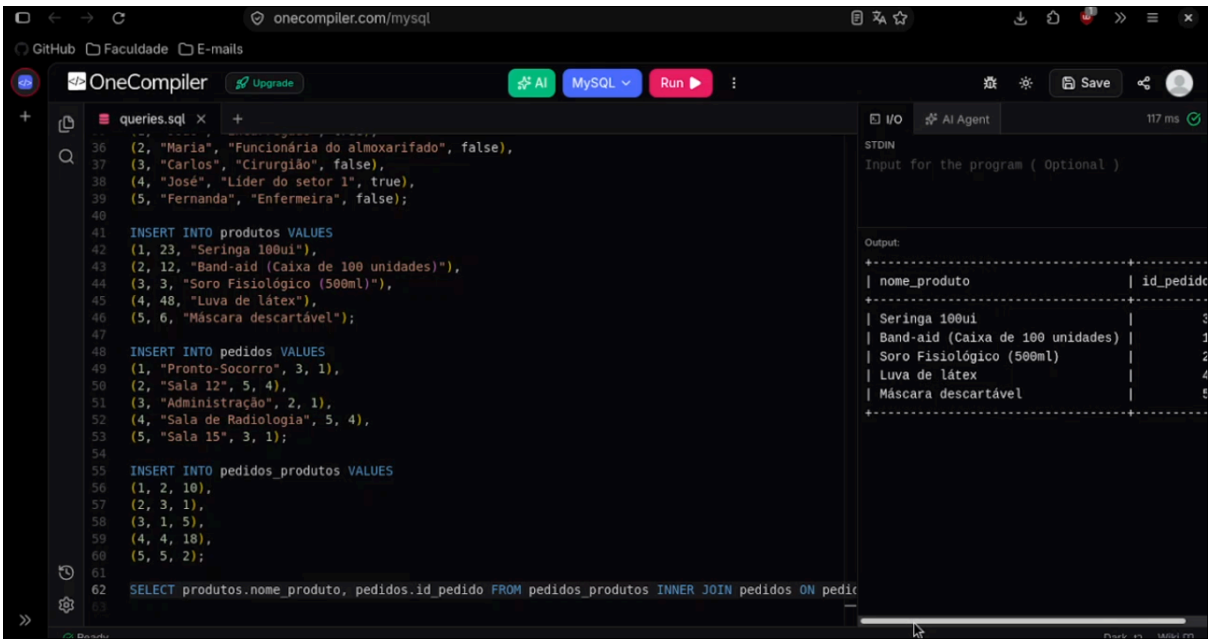



onecompiler.com/mysql
GitHub
Faculdade
E-mails
OneCompiler
Upgrade
AI
MySQL
Run
Save
queries.sql
(1, 23, "Seringa 100ui"),
(2, 12, "Band-aid (Caixa de 100 unidades)"),
(3, 3, "Soro Fisiológico (500ml)"),
(4, 48, "Luva de látex"),
(5, 6, "Máscara descartável");
INSERT INTO pedidos VALUES
(1, "Pronto-Socorro", 3, 1),
(2, "Sala 12", 5, 4),
(3, "Administração", 2, 1),
(4, "Sala de Radiologia", 5, 4),
(5, "Sala 15", 3, 1);
INSERT INTO pedidos_produtos VALUES
(1, 2, 10),
(2, 3, 1),
(3, 1, 5),
(4, 4, 18),
(5, 5, 2);
SELECT * FROM funcionarios;
-- SELECT * FROM produtos WHERE qtd_produto < 10;
-- SELECT * FROM pedidos_produtos ORDER BY qtd_solicitada;
-- SELECT MAX(qtd_produto) FROM produtos;
-- SELECT produtos.nome_produto, pedidos.id_pedido FROM pedidos_produtos INNER JOIN pedidos ON pe
I/O
AI Agent
113 ms
STDIN
Input for the program (Optional)
Output:
| id_funcionario | nome_funcionario | cargo
| 1 | João | Encarreg
| 2 | Maria | Funcioná
| 3 | Carlos | Cirurgiã
| 4 | José | Líder do
| 5 | Fernanda | Enfermei
Ready
Dark
Wiki


onecompiler.com/mysql
GitHub
Faculdade
E-mails
OneCompiler
Upgrade
AI
MySQL
Run
Save
queries.sql
INSERT INTO produtos VALUES
(1, 23, "Seringa 100ui"),
(2, 12, "Band-aid (Caixa de 100 unidades)"),
(3, 3, "Soro Fisiológico (500ml)"),
(4, 48, "Luva de látex"),
(5, 6, "Máscara descartável");
INSERT INTO pedidos VALUES
(1, "Pronto-Socorro", 3, 1),
(2, "Sala 12", 5, 4),
(3, "Administração", 2, 1),
(4, "Sala de Radiologia", 5, 4),
(5, "Sala 15", 3, 1);
INSERT INTO pedidos_produtos VALUES
(1, 2, 10),
(2, 3, 1),
(3, 1, 5),
(4, 4, 18),
(5, 5, 2);
SELECT * FROM produtos WHERE qtd_produto < 10;
-- SELECT * FROM pedidos_produtos ORDER BY qtd_solicitada;
-- SELECT MAX(qtd_produto) FROM produtos;
-- SELECT produtos.nome_produto, pedidos.id_pedido FROM pedidos_produtos INNER JOIN pedidos ON pr
I/O
AI Agent
66 ms
STDIN
Input for the program (Optional)
Output:
uto | qtd_produto | nome_produto
3 | 3 | Soro Fisiológico (500ml)
5 | 6 | Máscara descartável
Ready
Dark
Wiki


onecompiler.com/mysql
GitHub
Faculdade
E-mails
OneCompiler
Upgrade
AI
MySQL
Run
Save
queries.sql
(4, "José", "Líder do setor 1", true),
(5, "Fernanda", "Enfermeira", false);
INSERT INTO produtos VALUES
(1, 23, "Seringa 100ui"),
(2, 12, "Band-aid (Caixa de 100 unidades)"),
(3, 3, "Soro Fisiológico (500ml)"),
(4, 48, "Luva de látex"),
(5, 6, "Máscara descartável");
INSERT INTO pedidos VALUES
(1, "Pronto-Socorro", 3, 1),
(2, "Sala 12", 5, 4),
(3, "Administração", 2, 1),
(4, "Sala de Radiologia", 5, 4),
(5, "Sala 15", 3, 1);
INSERT INTO pedidos_produtos VALUES
(1, 2, 10),
(2, 3, 1),
(3, 1, 5),
(4, 4, 18),
(5, 5, 2);
SELECT * FROM pedidos_produtos ORDER BY qtd_solicitada;
-- SELECT MAX(qtd_produto) FROM produtos;
-- SELECT produtos.nome_produto, pedidos.id_pedido FROM pedidos_produtos INNER JOIN pedidos ON
I/O
AI Agent
104 ms
STDIN
Input for the program (Optional)
Output:
id_pedido	id_produto	qtd_solicitada
2	3	1
5	5	2
3	1	5
1	2	10
4	4	18
Ready
Dark
Wiki





CONCLUSÃO

Ao realizar-se esse projeto, foi possível colocar em prática os conhecimentos adquiridos durante a disciplina, planejando e organizando as informações ao estruturar as tabelas, fazer o relacionamento entre elas e consultas necessárias. Conclui-se, dessa forma, que os objetivos propostos foram alcançados por meio da aplicação do código e a estrutura adequada foi compreendida.

UTILIZAÇÃO DE IA:

Após algumas tentativas de organizar a sintaxe que estava acarretando em erro, copiou-se e colou-se o código inteiro na Inteligência Artificial (IA), a fim de revisar a explicação e entender o erro de relacionamento. Após esse estudo, adaptou-se o que já se tinha e utilizou-se o CONSTRAINT FOREIGN KEY para garantir uma declaração correta das chaves estrangeiras.

A IA foi utilizada conforme o quadro abaixo:

Data	Modelo utilizado	Prompt Utilizado	Onde foi usado
10/10/2026	GEMINI	Solicitamos a análise do código SQL de criação de tabelas que continha falhas de relacionamentos e de sintaxe entre as chaves primárias e estrangeiras.	Estrutura inicial do modelo do banco de dados do projeto

Ao terminar a estrutura do código em SQL, solicitou-se, mais uma vez, a IA, para revisar a sintaxe e identificar possíveis erros da construção. A resposta deu-se como positiva, entretanto, como solicitado, retornou-se algumas correções a serem feitas, como: i. O código tinha sido criado com colunas de texto: responsavel_autorizar VARCHAR e solicitante VARCHAR, sucedendo em redundância; ii. A criação da tabela “pedidos” esperava exatamente 4 colunas e no INSERT INTO pedidos passava 6 valores e estava sendo inserindo textos onde o banco esperava receber números inteiros; iii. A sintaxe estava com redundância desnecessária ao declarar-se as PRIMARY KEYS juntas dos comandos UNIQUE e NOT NULL. As chaves primárias já aplicam essas regras.

Data	Modelo utilizado	Prompt Utilizado	Onde foi usado:
11/10/2026	Gemini	Solicitamos a IA para revisar o código final e apontar os possíveis erros e/ou correções a serem feitas.	Estrutura completa do código.